

AP_HAL_Empty Line-Commented Source

Generated on 2026-05-01 from ardupilot/libraries/AP_HAL_Empty

Every non-include source line is followed by a comment. #include lines are preserved without added comments.

File Index

- AP_HAL_Empty.h
- AP_HAL_Empty_Namespace.h
- AP_HAL_Empty_Private.h
- AnalogIn.cpp
- AnalogIn.h
- DSP.h
- Flash.h
- GPIO.cpp
- GPIO.h
- HAL_Empty_Class.cpp
- HAL_Empty_Class.h
- I2CDevice.h
- OpticalFlow.h
- RCInput.cpp
- RCInput.h
- RCOOutput.cpp
- RCOOutput.h
- SPIDevice.h
- Scheduler.h
- Semaphores.cpp
- Semaphores.h
- Storage.cpp
- Storage.h
- UARTDriver.cpp
- UARTDriver.h
- Util.h
- WSPIDevice.h

AP_HAL_Empty.h

Path: ardupilot/libraries/AP_HAL_Empty/AP_HAL_Empty.h

```
1: #pragma once
   // Comment: Prevents this header from being included more than once in a single translation unit.
2:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: /* Your layer exports should depend on AP_HAL.h ONLY. */
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
4: #include <AP_HAL/AP_HAL.h>
5:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
6: /**
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
7:  * Umbrella header for AP_HAL_Empty module.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
8:  * The module header exports singleton instances which must conform the
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
9:  * AP_HAL::HAL interface. It may only expose implementation details (class
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
10:  * names, headers) via the Empty namespace.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
11:  * The class implementing AP_HAL::HAL should be called HAL_Empty and exist
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
12:  * in the global namespace. There should be a single const instance of the
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
13:  * HAL_Empty class called AP_HAL_Empty, instantiated in the HAL_Empty_Class.cpp
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
14:  * and exported as 'extern const HAL_Empty AP_HAL_Empty;' in HAL_Empty_Class.h
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
15:  *
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
16:  * All declaration and compilation should be guarded by CONFIG_HAL_BOARD macros.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
17:  * In this case, we're using CONFIG_HAL_BOARD == HAL_BOARD_EMPTY.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
```

```

18:  * When creating a new HAL, declare a new HAL_BOARD_ in AP_HAL/AP_HAL_Boards.h
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
19:  */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
20:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: #include "HAL_Empty_Class.h"

```

AP_HAL_Empty_Namespace.h

Path: ardupilot/libraries/AP_HAL_Empty/AP_HAL_Empty_Namespace.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: namespace Empty {
    // Comment: Opens a namespace scope so these declarations are grouped under the HAL-specific namespace.
4:     class AnalogIn;
    // Comment: Declares class AnalogIn;, part of the Empty HAL interface implementation.
5:     class AnalogSource;
    // Comment: Declares class AnalogSource;, part of the Empty HAL interface implementation.
6:     class DigitalSource;
    // Comment: Declares class DigitalSource;, part of the Empty HAL interface implementation.
7:     class DSP;
    // Comment: Declares class DSP;, part of the Empty HAL interface implementation.
8:     class GPIO;
    // Comment: Declares class GPIO;, part of the Empty HAL interface implementation.
9:     class I2CDevice;
    // Comment: Declares class I2CDevice;, part of the Empty HAL interface implementation.
10:    class I2CDeviceManager;
    // Comment: Declares class I2CDeviceManager;, part of the Empty HAL interface implementation.
11:    class OpticalFlow;
    // Comment: Declares class OpticalFlow;, part of the Empty HAL interface implementation.
12:    class RCInput;
    // Comment: Declares class RCInput;, part of the Empty HAL interface implementation.
13:    class RCOutput;
    // Comment: Declares class RCOutput;, part of the Empty HAL interface implementation.
14:    class Scheduler;
    // Comment: Declares class Scheduler;, part of the Empty HAL interface implementation.
15:    class Semaphore;
    // Comment: Declares class Semaphore;, part of the Empty HAL interface implementation.
16:    class SPIDevice;
    // Comment: Declares class SPIDevice;, part of the Empty HAL interface implementation.
17:    class SPIDeviceDriver;
    // Comment: Declares class SPIDeviceDriver;, part of the Empty HAL interface implementation.
18:    class SPIDeviceManager;
    // Comment: Declares class SPIDeviceManager;, part of the Empty HAL interface implementation.
19:    class WSPIDevice;
    // Comment: Declares class WSPIDevice;, part of the Empty HAL interface implementation.
20:    class WSPIDeviceManager;
    // Comment: Declares class WSPIDeviceManager;, part of the Empty HAL interface implementation.
21:    class Storage;
    // Comment: Declares class Storage;, part of the Empty HAL interface implementation.
22:    class UARTDriver;
    // Comment: Declares class UARTDriver;, part of the Empty HAL interface implementation.
23:    class Util;
    // Comment: Declares class Util;, part of the Empty HAL interface implementation.
24:    class Flash;
    // Comment: Declares class Flash;, part of the Empty HAL interface implementation.
25: }
    // Comment: Closes the current scope.

```

AP_HAL_Empty_Private.h

Path: ardupilot/libraries/AP_HAL_Empty/AP_HAL_Empty_Private.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: /* Umbrella header for all private headers of the AP_HAL_Empty module.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
4:  * Only import this header from inside AP_HAL_Empty
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
5:  */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
6:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.

```

```

7: #include "AnalogIn.h"
8: #include "GPIO.h"
9: #include "I2CDevice.h"
10: #include "OpticalFlow.h"
11: #include "RCInput.h"
12: #include "RCOutput.h"
13: #include "Scheduler.h"
14: #include "Semaphores.h"
15: #include "SPIDevice.h"
16: #include "WSPIDevice.h"
17: #include "Storage.h"
18: #include "UARTDriver.h"
19: #include "Util.h"
20: #include "Flash.h"
21: #include "DSP.h"

```

AnalogIn.cpp

Path: ardupilot/libraries/AP_HAL_Empty/AnalogIn.cpp

```

1: #include "AnalogIn.h"
2:
3: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
4: using namespace Empty;
5: // Comment: Brings the named namespace into local lookup so implementation code can use shorter type
6: // names.
7:
8: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
9: AnalogSource::AnalogSource(float v) :
10: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
11: // interface.
12: _v(v)
13: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
14: // interface.
15: {}
16: // Comment: Provides an empty implementation body because the stub has no work to perform.
17:
18: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: float AnalogSource::read_average() {
20: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
21:     return _v;
22: // Comment: Returns the computed or stored value required by the current AP_HAL method.
23: }
24: // Comment: Closes the current scope.
25:
26: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
27: float AnalogSource::voltage_average() {
28: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
29:     return 5.0f * _v / 1024.0f;
30: // Comment: Returns the fixed simulated board voltage used by this empty analog implementation.
31: }
32: // Comment: Closes the current scope.
33:
34: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
35: float AnalogSource::voltage_latest() {
36: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
37:     return 5.0f * _v / 1024.0f;
38: // Comment: Returns the fixed simulated board voltage used by this empty analog implementation.
39: }
40: // Comment: Closes the current scope.
41:
42: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
43: float AnalogSource::read_latest() {
44: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
45:     return _v;
46: // Comment: Returns the computed or stored value required by the current AP_HAL method.
47: }
48: // Comment: Closes the current scope.
49:
50: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
51: bool AnalogSource::set_pin(uint8_t p) {
52: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
53:     return true;
54: // Comment: Reports success for an operation that the stub accepts without touching real hardware.
55: }
56: // Comment: Closes the current scope.
57:
58: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
59: AnalogIn::AnalogIn()
60: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
61: // interface.
62: {}
63: // Comment: Provides an empty implementation body because the stub has no work to perform.
64:
65: // Comment: Blank separator line; keeps related declarations or statements visually grouped.

```

```

32: void AnalogIn::init()
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
33: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
34:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
35: AP_HAL::AnalogSource* AnalogIn::channel(int16_t n) {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
36:     return NEW_NOTHROW AnalogSource(1.11);
    // Comment: Allocates and returns a stub object without throwing an exception on allocation failure.
37: }
    // Comment: Closes the current scope.
38:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
39: float AnalogIn::board_voltage(void)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
40: {
    // Comment: Opens the body for the preceding declaration or control statement.
41:     return 5.0f;
    // Comment: Returns the fixed simulated board voltage used by this empty analog implementation.
42: }
    // Comment: Closes the current scope.

```

AnalogIn.h

Path: ardupilot/libraries/AP_HAL_Empty/AnalogIn.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::AnalogSource : public AP_HAL::AnalogSource {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
    // Comment: Sets the C++ member access level to public for following class members.
7:     AnalogSource(float v);
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL inte
    // rface.
8:     float read_average() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
9:     float read_latest() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:     bool set_pin(uint8_t p) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
11:     float voltage_average() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
12:     float voltage_latest() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
13:     float voltage_average_ratiometric() override { return voltage_average(); }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
14: private:
    // Comment: Sets the C++ member access level to private for following class members.
15:     float _v;
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
16: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.
17:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
18: class Empty::AnalogIn : public AP_HAL::AnalogIn {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
19: public:
    // Comment: Sets the C++ member access level to public for following class members.
20:     AnalogIn();
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL inte
    // rface.
21:     void init() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
22:     AP_HAL::AnalogSource* channel(int16_t n) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
23:     float board_voltage(void) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
24: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

DSP.h

Path: ardupilot/libraries/AP_HAL_Empty/DSP.h

```

1: /*
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
2: * This file is free software: you can redistribute it and/or modify it
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
3: * under the terms of the GNU General Public License as published by the
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
4: * Free Software Foundation, either version 3 of the License, or
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
5: * (at your option) any later version.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
6: *
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
7: * This file is distributed in the hope that it will be useful, but
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
8: * WITHOUT ANY WARRANTY; without even the implied warranty of
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
9: * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
10: * See the GNU General Public License for more details.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
11: *
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
12: * You should have received a copy of the GNU General Public License along
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
13: * with this program. If not, see <http://www.gnu.org/licenses/>.
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
14: *
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
15: * Code by Andy Piper
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
16: */
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
17: #pragma once
   // Comment: Prevents this header from being included more than once in a single translation unit.
18:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: #include "AP_HAL_Empty.h"
20:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: class Empty::DSP : public AP_HAL::DSP {
   // Comment: Declares class Empty, part of the Empty HAL interface implementation.
22: #if HAL_WITH_DSP
   // Comment: Starts a preprocessor condition so this code is compiled only when the build configuration matches.
23: public:
   // Comment: Sets the C++ member access level to public for following class members.
24:     virtual FFTWindowState* fft_init(uint16_t window_size, uint16_t sample_rate, uint8_t sliding_window_size) override { return nullptr; }
   // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
25:     virtual void fft_start(FFTWindowState* state, FloatBuffer& samples, uint16_t advance) override {
    }
   // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
26:     virtual uint16_t fft_analyse(FFTWindowState* state, uint16_t start_bin, uint16_t end_bin, float noise_att_cutoff) override { return 0; }
   // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
27: protected:
   // Comment: Sets the C++ member access level to protected for following class members.
28:     virtual void vector_max_float(const float* vin, uint16_t len, float* maxValue, uint16_t* maxIndex) const override {}
   // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
29:     virtual void vector_scale_float(const float* vin, float scale, float* vout, uint16_t len) const override {}
   // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.

```

```

30:     virtual float vector_mean_float(const float* vin, uint16_t len) const override { return 0.0f; };
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
31:     virtual void vector_add_float(const float* vin1, const float* vin2, float* vout, uint16_t len) c
    onst override {};
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
32: #endif // HAL_WITH_DSP
    // Comment: Ends the matching preprocessor conditional block.
33: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

Flash.h

Path: ardupilot/libraries/AP_HAL_Empty/Flash.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::Flash : public AP_HAL::Flash {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
    // Comment: Sets the C++ member access level to public for following class members.
7:     uint32_t getpageaddr(uint32_t page) override { return 0; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
8:     uint32_t getpagesize(uint32_t page) override { return 0; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
9:     uint32_t getnumpages(void) override { return 0; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
10:     bool erasepage(uint32_t page) override { return false; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
11:     bool write(uint32_t addr, const void *buf, uint32_t count) override { return false; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
12:     void keep_unlocked(bool set) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    rk here.
13:     bool ispageerased(uint32_t page) override { return false; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
14: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

GPIO.cpp

Path: ardupilot/libraries/AP_HAL_Empty/GPIO.cpp

```

1:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include "GPIO.h"
3:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
4: using namespace Empty;
    // Comment: Brings the named namespace into local lookup so implementation code can use shorter type
    names.
5:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
6: GPIO::GPIO()
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
7: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
8:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
9: void GPIO::init()
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
10: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
11:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
12: void GPIO::pinMode(uint8_t pin, uint8_t output)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
13: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
14:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
15: uint8_t GPIO::read(uint8_t pin) {

```

```

    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
16:     return 0;
    // Comment: Returns a neutral zero value, typically meaning no data, no size, or no available hardware state.
17: }
    // Comment: Closes the current scope.
18:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: void GPIO::write(uint8_t pin, uint8_t value)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
20: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
21:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
22: void GPIO::toggle(uint8_t pin)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
23: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
24:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
25: /* Alternative interface: */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
26: AP_HAL::DigitalSource* GPIO::channel(uint16_t n) {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
27:     return NEW_NOTHROW DigitalSource(0);
    // Comment: Allocates and returns a stub object without throwing an exception on allocation failure.
28: }
    // Comment: Closes the current scope.
29:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
30: bool GPIO::usb_connected(void)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
31: {
    // Comment: Opens the body for the preceding declaration or control statement.
32:     return false;
    // Comment: Reports failure or absence of data for this stubbed hardware operation.
33: }
    // Comment: Closes the current scope.
34:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
35: DigitalSource::DigitalSource(uint8_t v) :
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
36:     _v(v)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
37: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
38:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
39: void DigitalSource::mode(uint8_t output)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
40: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
41:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
42: uint8_t DigitalSource::read() {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
43:     return _v;
    // Comment: Returns the computed or stored value required by the current AP_HAL method.
44: }
    // Comment: Closes the current scope.
45:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
46: void DigitalSource::write(uint8_t value) {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
47:     _v = value;
    // Comment: Assigns or updates local state used by this stub implementation.
48: }
    // Comment: Closes the current scope.
49:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
50: void DigitalSource::toggle() {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
51:     _v = !_v;
    // Comment: Assigns or updates local state used by this stub implementation.
52: }
    // Comment: Closes the current scope.

```

GPIO.h

Path: ardupilot/libraries/AP_HAL_Empty/GPIO.h

```
1: #pragma once
  // Comment: Prevents this header from being included more than once in a single translation unit.
2:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::GPIO : public AP_HAL::GPIO {
  // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
  // Comment: Sets the C++ member access level to public for following class members.
7:     GPIO();
  // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
8:     void    init() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
9:     void    pinMode(uint8_t pin, uint8_t output) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:    uint8_t read(uint8_t pin) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
11:    void    write(uint8_t pin, uint8_t value) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
12:    void    toggle(uint8_t pin) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
13:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
14:    /* Alternative interface: */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
15:    AP_HAL::DigitalSource* channel(uint16_t n) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
16:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
17:    /* return true if USB cable is connected */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
18:    bool    usb_connected(void) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
19: };
  // Comment: Closes a class declaration and terminates it with the required semicolon.
20:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: class Empty::DigitalSource : public AP_HAL::DigitalSource {
  // Comment: Declares class Empty, part of the Empty HAL interface implementation.
22: public:
  // Comment: Sets the C++ member access level to public for following class members.
23:     DigitalSource(uint8_t v);
  // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
24:     void    mode(uint8_t output) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
25:     uint8_t read() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
26:     void    write(uint8_t value) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
27:     void    toggle() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
28: private:
  // Comment: Sets the C++ member access level to private for following class members.
29:     uint8_t _v;
  // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
30: };
  // Comment: Closes a class declaration and terminates it with the required semicolon.
```

HAL_Empty_Class.cpp

Path: ardupilot/libraries/AP_HAL_Empty/HAL_Empty_Class.cpp

```
1:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include <AP_HAL/AP_HAL.h>
3: #if CONFIG_HAL_BOARD == HAL_BOARD_EMPTY
  // Comment: Starts a preprocessor condition so this code is compiled only when the build configuration matches.
4:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: #include <assert.h>
6:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
7: #include "HAL_Empty_Class.h"
```



```

8: #include "AP_HAL_Empty_Private.h"
9:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
10: using namespace Empty;
  // Comment: Brings the named namespace into local lookup so implementation code can use shorter type
  names.
11:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
12: static UARTDriver serial0Driver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
13: static UARTDriver serial1Driver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
14: static UARTDriver serial2Driver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
15: static UARTDriver serial3Driver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
16: static SPIDeviceManager spiDeviceManager;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
17: static AnalogIn analogIn;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
18: static Storage storageDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
19: static GPIO gpioDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
20: static RCInput rcinDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
21: static RCOutput rcoutDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
22: static Scheduler schedulerInstance;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
23: static Util utilInstance;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
24: static OpticalFlow opticalFlowDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
25: static Flash flashDriver;
  // Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
26:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
27: HAL_Empty::HAL_Empty() :
  // Comment: Defines the Empty HAL constructor that wires driver objects into the AP_HAL base class.
28:     AP_HAL::HAL(
  // Comment: Starts the AP_HAL base-class constructor call with this HAL port's driver table.
29:         &serial0Driver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
30:         &serial1Driver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
31:         &serial2Driver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
32:         &serial3Driver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
33:         nullptr, /* no SERIAL4 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
34:         nullptr, /* no SERIAL5 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
35:         nullptr, /* no SERIAL6 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
36:         nullptr, /* no SERIAL7 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
37:         nullptr, /* no SERIAL8 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
38:         nullptr, /* no SERIAL9 */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
39:         &spiDeviceManager,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
40:         &analogIn,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
41:         &storageDriver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
42:         &serial0Driver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
43:         &gpioDriver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
44:         &rcinDriver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
45:         &rcoutDriver,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
46:         &schedulerInstance,
  // Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.

```

```

47:         &utilInstance,
// Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
48:         &opticalFlowDriver,
// Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
49:         &flashDriver,
// Comment: Passes either a stub driver pointer or a null slot into the AP_HAL constructor.
50:         nullptr) /* no DSP */
// Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
51: {}
// Comment: Provides an empty implementation body because the stub has no work to perform.
52:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
53: void HAL_Empty::run(int argc, char* const argv[], Callbacks* callbacks) const
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
54: {
// Comment: Opens the body for the preceding declaration or control statement.
55:     /* initialize all drivers and private members here.
// Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
56:     * up to the programmer to do this in the correct order.
// Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
57:     * Scheduler should likely come first. */
// Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
58:     scheduler->init();
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
59:     serial(0)->begin(115200);
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
60:     _member->init();
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
61:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
62:     callbacks->setup();
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
63:     scheduler->set_system_initialized();
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
64:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
65:     for (;;) {
// Comment: Starts a loop that repeats the following block over the requested range.
66:         callbacks->loop();
// Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
67:     }
// Comment: Closes the current scope.
68: }
// Comment: Closes the current scope.
69:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
70: static HAL_Empty hal_empty;
// Comment: Creates a file-local object that backs one of the singleton-style HAL driver instances.
71:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
72: const AP_HAL::HAL& AP_HAL::get_HAL() {
// Comment: Defines the public accessor that returns the global HAL instance as a const reference.
73:     return hal_empty;
// Comment: Returns the computed or stored value required by the current AP_HAL method.
74: }
// Comment: Closes the current scope.
75:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
76: AP_HAL::HAL& AP_HAL::get_HAL_mutable() {
// Comment: Defines the mutable accessor for code that needs non-const access to the HAL instance.
77:     return hal_empty;
// Comment: Returns the computed or stored value required by the current AP_HAL method.
78: }
// Comment: Closes the current scope.
79:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
80: #endif
// Comment: Ends the matching preprocessor conditional block.

```

HAL_Empty_Class.h

Path: ardupilot/libraries/AP_HAL_Empty/HAL_Empty_Class.h

```
1: #pragma once
```

```

    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include <AP_HAL/AP_HAL.h>
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: #include "AP_HAL_Empty_Namespace.h"
6:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
7: class HAL_Empty : public AP_HAL::HAL {
    // Comment: Declares class HAL_Empty, part of the Empty HAL interface implementation.
8: public:
    // Comment: Sets the C++ member access level to public for following class members.
9:     HAL_Empty();
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
10:     void run(int argc, char* const* argv, Callbacks* callbacks) const override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
11: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

I2CDevice.h

Path: ardupilot/libraries/AP_HAL_Empty/I2CDevice.h

```

1: /*
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
2:  * Copyright (C) 2015-2016 Intel Corporation. All rights reserved.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
3:  *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
4:  * This file is free software: you can redistribute it and/or modify it
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
5:  * under the terms of the GNU General Public License as published by the
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
6:  * Free Software Foundation, either version 3 of the License, or
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
7:  * (at your option) any later version.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
8:  *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
9:  * This file is distributed in the hope that it will be useful, but
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
10:  * WITHOUT ANY WARRANTY; without even the implied warranty of
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
11:  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
12:  * See the GNU General Public License for more details.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
13:  *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
14:  * You should have received a copy of the GNU General Public License along
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
15:  * with this program. If not, see <http://www.gnu.org/licenses/>.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
16:  */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
17: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
18:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: #include <inttypes.h>
20:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: #include <AP_HAL/HAL.h>
22: #include <AP_HAL/I2CDevice.h>
23:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
24: namespace Empty {

```

```

    // Comment: Opens a namespace scope so these declarations are grouped under the HAL-specific namespace.
25:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
26: class I2CDevice : public AP_HAL::I2CDevice {
    // Comment: Declares class I2CDevice, part of the Empty HAL interface implementation.
27: public:
    // Comment: Sets the C++ member access level to public for following class members.
28:     I2CDevice()
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
29:     {
    // Comment: Opens the body for the preceding declaration or control statement.
30:     }
    // Comment: Closes the current scope.
31:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
32:     virtual ~I2CDevice() { }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
33:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
34:     /* AP_HAL::I2CDevice implementation */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
35:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
36:     /* See AP_HAL::I2CDevice::set_address() */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
37:     void set_address(uint8_t address) override { }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
38:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
39:     /* See AP_HAL::I2CDevice::set_retries() */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
40:     void set_retries(uint8_t retries) override { }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
41:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
42:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
43:     /* AP_HAL::Device implementation */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
44:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
45:     /* See AP_HAL::Device::transfer() */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
46:     bool transfer(const uint8_t *send, uint32_t send_len,
    // Comment: Supplies one argument in a multi-line constructor or function call.
        uint8_t *recv, uint32_t recv_len) override
47:     // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
48:     {
    // Comment: Opens the body for the preceding declaration or control statement.
49:         return true;
    // Comment: Reports success for an operation that the stub accepts without touching real hardware.
50:     }
    // Comment: Closes the current scope.
51:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
52:     bool read_registers_multiple(uint8_t first_reg, uint8_t *recv,
    // Comment: Supplies one argument in a multi-line constructor or function call.
        uint32_t recv_len, uint8_t times) override
53:     // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
        interface.
54:     {
    // Comment: Opens the body for the preceding declaration or control statement.
55:         return true;
    // Comment: Reports success for an operation that the stub accepts without touching real hardware.
56:     }
    // Comment: Closes the current scope.
57:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
58:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
59:     /* See AP_HAL::Device::set_speed() */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.

```

```

60:     bool set_speed(enum AP_HAL::Device::Speed speed) override { return true; }
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
61:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
62:     /* See AP_HAL::Device::get_semaphore() */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
// tes.
63:     AP_HAL::Semaphore *get_semaphore() override { return nullptr; }
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
64:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
65:     /* See AP_HAL::Device::register_periodic_callback() */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
// tes.
66:     AP_HAL::Device::PeriodicHandle register_periodic_callback(
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
67:         uint32_t period_usec, AP_HAL::Device::PeriodicCb) override
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
68:     {
// Comment: Opens the body for the preceding declaration or control statement.
69:         return nullptr;
// Comment: Returns no object or no callback handle because the Empty HAL does not provide this hard
// ware service.
70:     }
// Comment: Closes the current scope.
71:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
72:     /* See Device::adjust_periodic_callback() */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
// tes.
73:     virtual bool adjust_periodic_callback(
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
74:         AP_HAL::Device::PeriodicHandle h, uint32_t period_usec) override
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
75:     {
// Comment: Opens the body for the preceding declaration or control statement.
76:         return true;
// Comment: Reports success for an operation that the stub accepts without touching real hardware.
77:     }
// Comment: Closes the current scope.
78: };
// Comment: Closes a class declaration and terminates it with the required semicolon.
79:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
80: class I2CDeviceManager : public AP_HAL::I2CDeviceManager {
// Comment: Declares class I2CDeviceManager, part of the Empty HAL interface implementation.
81: public:
// Comment: Sets the C++ member access level to public for following class members.
82:     I2CDeviceManager() { }
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
83:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
84:     /* AP_HAL::I2CDeviceManager implementation */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
// tes.
85:     AP_HAL::I2CDevice *get_device_ptr(uint8_t bus, uint8_t address,
// Comment: Supplies one argument in a multi-line constructor or function call.
86:         uint32_t bus_clock=400000,
// Comment: Supplies one argument in a multi-line constructor or function call.
87:         bool use_smbus = false,
// Comment: Supplies one argument in a multi-line constructor or function call.
88:         uint32_t timeout_ms=4) override
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
// interface.
89:     {
// Comment: Opens the body for the preceding declaration or control statement.
90:         return nullptr;
// Comment: Returns no object or no callback handle because the Empty HAL does not provide this hard
// ware service.
91:     }
// Comment: Closes the current scope.
92: };
// Comment: Closes a class declaration and terminates it with the required semicolon.
93:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
94: }
// Comment: Closes the current scope.

```

OpticalFlow.h

Path: ardupilot/libraries/AP_HAL_Empty/OpticalFlow.h

```
1: /*
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
2:   This program is free software: you can redistribute it and/or modify
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
3:   it under the terms of the GNU General Public License as published by
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
4:   the Free Software Foundation, either version 3 of the License, or
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
5:   (at your option) any later version.
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
6:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
7:   This program is distributed in the hope that it will be useful,
   // Comment: Supplies one argument in a multi-line constructor or function call.
8:   but WITHOUT ANY WARRANTY; without even the implied warranty of
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
9:   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
10:  GNU General Public License for more details.
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
11:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
12:  You should have received a copy of the GNU General Public License
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
13:  along with this program. If not, see <http://www.gnu.org/licenses/>.
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
14: */
   // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
15: #pragma once
   // Comment: Prevents this header from being included more than once in a single translation unit.
16:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
17: class Empty::OpticalFlow : public AP_HAL::OpticalFlow {
   // Comment: Declares class Empty, part of the Empty HAL interface implementation.
18: public:
   // Comment: Sets the C++ member access level to public for following class members.
19:   void init() override { }
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
20:   bool read(Data_Frame& frame) override { return false; }
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
21:   void push_gyro(float gyro_x, float gyro_y, float dt) override { };
   // Comment: Declares an implementation of a virtual AP_HAL interface method.
22:   void push_gyro_bias(float gyro_bias_x, float gyro_bias_y) override { }
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
23: };
   // Comment: Closes a class declaration and terminates it with the required semicolon.
```

RCInput.cpp

Path: ardupilot/libraries/AP_HAL_Empty/RCInput.cpp

```
1:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include "RCInput.h"
3:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
4: using namespace Empty;
   // Comment: Brings the named namespace into local lookup so implementation code can use shorter type names.
5: RCInput::RCInput()
   // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
6: {}
   // Comment: Provides an empty implementation body because the stub has no work to perform.
7:
   // Comment: Blank separator line; keeps related declarations or statements visually grouped.
```

```

8: void RCInput::init()
  // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++
  // interface.
9: {}
  // Comment: Provides an empty implementation body because the stub has no work to perform.
10:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
11: bool RCInput::new_input() {
  // Comment: Opens the body where this class, method, conditional, or loop is implemented.
12:     return false;
  // Comment: Reports failure or absence of data for this stubbed hardware operation.
13: }
  // Comment: Closes the current scope.
14:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
15: uint8_t RCInput::num_channels() {
  // Comment: Opens the body where this class, method, conditional, or loop is implemented.
16:     return 0;
  // Comment: Returns a neutral zero value, typically meaning no data, no size, or no available hardware state.
17: }
  // Comment: Closes the current scope.
18:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: uint16_t RCInput::read(uint8_t chan) {
  // Comment: Opens the body where this class, method, conditional, or loop is implemented.
20:     if (chan == 2) return 900; /* throttle should be low, for safety */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
21:     else return 1500;
  // Comment: Handles the alternative path when the preceding condition was not met.
22: }
  // Comment: Closes the current scope.
23:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
24: uint8_t RCInput::read(uint16_t* periods, uint8_t len) {
  // Comment: Opens the body where this class, method, conditional, or loop is implemented.
25:     for (uint8_t i = 0; i < len; i++){
  // Comment: Starts a loop that repeats the following block over the requested range.
26:         if (i == 2) periods[i] = 900;
  // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
27:         else periods[i] = 1500;
  // Comment: Handles the alternative path when the preceding condition was not met.
28:     }
  // Comment: Closes the current scope.
29:     return len;
  // Comment: Returns the computed or stored value required by the current AP_HAL method.
30: }
  // Comment: Closes the current scope.
31:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.

```

RCInput.h

Path: ardupilot/libraries/AP_HAL_Empty/RCInput.h

```

1: #pragma once
  // Comment: Prevents this header from being included more than once in a single translation unit.
2:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::RCInput : public AP_HAL::RCInput {
  // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
  // Comment: Sets the C++ member access level to public for following class members.
7:     RCInput();
  // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
8:     void init() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
9:     bool new_input() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:    uint8_t num_channels() override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
11:    uint16_t read(uint8_t ch) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
12:    uint8_t read(uint16_t* periods, uint8_t len) override;
  // Comment: Declares an implementation of a virtual AP_HAL interface method.
13:    virtual const char *protocol() const override { return "Empty"; }
  // Comment: Keeps the AP_HAL_Empty implementation structurally compatible with the corresponding C++ interface.
14:

```

```

    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
15: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

RCOutput.cpp

Path: ardupilot/libraries/AP_HAL_Empty/RCOutput.cpp

```

1:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include "RCOutput.h"
3: #include <AP_Math/AP_Math.h>
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: using namespace Empty;
    // Comment: Brings the named namespace into local lookup so implementation code can use shorter type
    names.
6:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
7: void RCOutput::init() {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    rk here.
8:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
9: void RCOutput::set_freq(uint32_t chmask, uint16_t freq_hz) {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    rk here.
10:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
11: uint16_t RCOutput::get_freq(uint8_t chan) {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
12:     return 50;
    // Comment: Returns the fixed default RC output frequency of 50 Hz.
13: }
    // Comment: Closes the current scope.
14:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
15: void RCOutput::enable_ch(uint8_t chan)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
16: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
17:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
18: void RCOutput::disable_ch(uint8_t chan)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
19: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
20:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: void RCOutput::write(uint8_t chan, uint16_t period_us)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
22: {
    // Comment: Opens the body for the preceding declaration or control statement.
23:     if (chan < ARRAY_SIZE(value)) {
    // Comment: Checks a condition before choosing whether to run the following block.
24:         value[chan] = period_us;
    // Comment: Assigns or updates local state used by this stub implementation.
25:     }
    // Comment: Closes the current scope.
26: }
    // Comment: Closes the current scope.
27:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
28: uint16_t RCOutput::read(uint8_t chan)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
29: {
    // Comment: Opens the body for the preceding declaration or control statement.
30:     if (chan < ARRAY_SIZE(value)) {
    // Comment: Checks a condition before choosing whether to run the following block.
31:         return value[chan];
    // Comment: Returns the computed or stored value required by the current AP_HAL method.
32:     }
    // Comment: Closes the current scope.
33:     return 900;
    // Comment: Returns a low PWM value, used as a safe fallback for throttle-like outputs.
34: }
    // Comment: Closes the current scope.
35:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
36: void RCOutput::read(uint16_t* period_us, uint8_t len)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++

```



```

    interface.
37: {
    // Comment: Opens the body for the preceding declaration or control statement.
38:     len = MIN(len, ARRAY_SIZE(value));
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
39:     memcpy(period_us, value, len*sizeof(value[0]));
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
40: }
    // Comment: Closes the current scope.
41:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.

```

RCOutput.h

Path: ardupilot/libraries/AP_HAL_Empty/RCOutput.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::RCOutput : public AP_HAL::RCOutput {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6:     void    init() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
7:     void    set_freq(uint32_t chmask, uint16_t freq_hz) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
8:     uint16_t get_freq(uint8_t ch) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
9:     void    enable_ch(uint8_t ch) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:    void    disable_ch(uint8_t ch) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
11:    void    write(uint8_t ch, uint16_t period_us) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
12:    uint16_t read(uint8_t ch) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
13:    void    read(uint16_t* period_us, uint8_t len) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
14:    void    cork(void) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
15:    void    push(void) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
16: private:
    // Comment: Sets the C++ member access level to private for following class members.
17:     uint16_t value[16];
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
18: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

SPIDevice.h

Path: ardupilot/libraries/AP_HAL_Empty/SPIDevice.h

```

1: /*
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
2:  * Copyright (C) 2015-2016 Intel Corporation. All rights reserved.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
3:  *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
4:  * This file is free software: you can redistribute it and/or modify it
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
5:  * under the terms of the GNU General Public License as published by the
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
6:  * Free Software Foundation, either version 3 of the License, or
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
7:  * (at your option) any later version.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
8:  *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.

```

```

9:  * This file is distributed in the hope that it will be useful, but
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
10:  * WITHOUT ANY WARRANTY; without even the implied warranty of
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
11:  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
12:  * See the GNU General Public License for more details.
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
13:  *
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
14:  * You should have received a copy of the GNU General Public License along
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
15:  * with this program. If not, see <http://www.gnu.org/licenses/>.
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
16:  */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
17: #pragma once
  // Comment: Prevents this header from being included more than once in a single translation unit.
18:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: #include <inttypes.h>
20:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: #include <AP_HAL/HAL.h>
22: #include <AP_HAL/SPIDevice.h>
23:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
24: #include "Semaphores.h"
25:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
26: namespace Empty {
  // Comment: Opens a namespace scope so these declarations are grouped under the HAL-specific namespa
  ce.
27:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
28: class SPIDevice : public AP_HAL::SPIDevice {
  // Comment: Declares class SPIDevice, part of the Empty HAL interface implementation.
29: public:
  // Comment: Sets the C++ member access level to public for following class members.
30:     SPIDevice()
  // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
  interface.
31:     {
  // Comment: Opens the body for the preceding declaration or control statement.
32:     }
  // Comment: Closes the current scope.
33:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
34:     virtual ~SPIDevice() { }
  // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
  interface.
35:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
36:     /* AP_HAL::Device implementation */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
37:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
38:     /* See AP_HAL::Device::set_speed() */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
39:     bool set_speed(AP_HAL::Device::Speed speed) override
  // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
  interface.
40:     {
  // Comment: Opens the body for the preceding declaration or control statement.
41:         return true;
  // Comment: Reports success for an operation that the stub accepts without touching real hardware.
42:     }
  // Comment: Closes the current scope.
43:
  // Comment: Blank separator line; keeps related declarations or statements visually grouped.
44:     /* See AP_HAL::Device::transfer() */
  // Comment: Source comment from the original file; documents licensing, intent, or implementation no
  tes.
45:     bool transfer(const uint8_t *send, uint32_t send_len,

```

```

46: // Comment: Supplies one argument in a multi-line constructor or function call.
    uint8_t *recv, uint32_t recv_len) override
47: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    {
48: // Comment: Opens the body for the preceding declaration or control statement.
    return true;
49: // Comment: Reports success for an operation that the stub accepts without touching real hardware.
    }
50: // Comment: Closes the current scope.

51: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
    /* See AP_HAL::SPIDevice::transfer_full duplex() */
52: // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
    bool transfer_full duplex(const uint8_t *send, uint8_t *recv,
53: // Comment: Supplies one argument in a multi-line constructor or function call.
    uint32_t len) override
54: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    {
55: // Comment: Opens the body for the preceding declaration or control statement.
    return true;
56: // Comment: Reports success for an operation that the stub accepts without touching real hardware.
    }
57: // Comment: Closes the current scope.

58: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
    /* See AP_HAL::Device::get_semaphore() */
59: // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
    AP_HAL::Semaphore *get_semaphore() override
60: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    {
61: // Comment: Opens the body for the preceding declaration or control statement.
    return _ semaphore;
62: // Comment: Returns the computed or stored value required by the current AP_HAL method.
    }
63: // Comment: Closes the current scope.

64: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
    /* See AP_HAL::Device::register_periodic_callback() */
65: // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
    AP_HAL::Device::PeriodicHandle register_periodic_callback(
66: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    uint32_t period_usec, AP_HAL::Device::PeriodicCb) override
67: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    {
68: // Comment: Opens the body for the preceding declaration or control statement.
    return nullptr;
69: // Comment: Returns no object or no callback handle because the Empty HAL does not provide this hard
    ware service.
    }
70: // Comment: Closes the current scope.

71: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
private:
72: // Comment: Sets the C++ member access level to private for following class members.
    Semaphore _ semaphore;
73: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
};
74: // Comment: Closes a class declaration and terminates it with the required semicolon.

75: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
class SPIDeviceManager : public AP_HAL::SPIDeviceManager {
76: // Comment: Declares class SPIDeviceManager, part of the Empty HAL interface implementation.
public:
77: // Comment: Sets the C++ member access level to public for following class members.
    SPIDeviceManager() { }
78: // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
    AP_HAL::SPIDevice *get_device_ptr(const char *name) override {
79: // Comment: Opens the body where this class, method, conditional, or loop is implemented.
    return NEW_NOTHROW SPIDevice();
80: // Comment: Allocates and returns a stub object without throwing an exception on allocation failure.
    }
81: // Comment: Closes the current scope.
};
82: // Comment: Closes a class declaration and terminates it with the required semicolon.

```

```

82: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
83: }
    // Comment: Closes the current scope.

```

Scheduler.h

Path: ardupilot/libraries/AP_HAL_Empty/Scheduler.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::Scheduler : public AP_HAL::Scheduler {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
    // Comment: Sets the C++ member access level to public for following class members.
7:     Scheduler() {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
8:     void init() override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
9:     void delay(uint16_t ms) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
10:    void delay_microseconds(uint16_t us) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
11:    void register_timer_process(AP_HAL::MemberProc) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
12:    void register_io_process(AP_HAL::MemberProc) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
13:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
14:    void register_timer_failsafe(AP_HAL::Proc, uint32_t period_us) override {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform work here.
15:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
16:    void set_system_initialized() override {}
    // Comment: Marks the HAL as initialized so higher-level code can proceed.
17:    bool is_system_initialized() override { return true; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
18:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
19:    void reboot(bool hold_in_bootloader) override { for (;;) }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
20:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

Semaphores.cpp

Path: ardupilot/libraries/AP_HAL_Empty/Semaphores.cpp

```

1:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include "Semaphores.h"
3:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
4: using namespace Empty;
    // Comment: Brings the named namespace into local lookup so implementation code can use shorter type names.
5:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
6: bool Semaphore::give() {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
7:     if (!_taken) {
    // Comment: Checks a condition before choosing whether to run the following block.
8:         _taken = false;
    // Comment: Assigns or updates local state used by this stub implementation.
9:         return true;
    // Comment: Reports success for an operation that the stub accepts without touching real hardware.
10:    } else {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
11:        return false;

```

```

12:     // Comment: Reports failure or absence of data for this stubbed hardware operation.
13: }
14: // Comment: Closes the current scope.
15: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
16: bool Semaphore::take(uint32_t timeout_ms) {
17:     // Comment: Opens the body where this class, method, conditional, or loop is implemented.
18:     return take_nonblocking();
19: }
20: // Comment: Returns the computed or stored value required by the current AP_HAL method.
21: // Comment: Closes the current scope.
22: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
23: bool Semaphore::take_nonblocking() {
24:     // Comment: Opens the body where this class, method, conditional, or loop is implemented.
25:     /* No synchronisation primitives to guarantee this is correct */
26:     // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
27:     if (!_taken) {
28:         // Comment: Checks a condition before choosing whether to run the following block.
29:         _taken = true;
30:         // Comment: Assigns or updates local state used by this stub implementation.
31:         return true;
32:     }
33:     // Comment: Reports success for an operation that the stub accepts without touching real hardware.
34:     } else {
35:         // Comment: Opens the body where this class, method, conditional, or loop is implemented.
36:         return false;
37:     }
38:     // Comment: Reports failure or absence of data for this stubbed hardware operation.
39: }
40: // Comment: Closes the current scope.
41: }
42: // Comment: Closes the current scope.

```

Semaphores.h

Path: ardupilot/libraries/AP_HAL_Empty/Semaphores.h

```

1: #pragma once
2: // Comment: Prevents this header from being included more than once in a single translation unit.
3: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
4: #include "AP_HAL_Empty.h"
5: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
6: class Empty::Semaphore : public AP_HAL::Semaphore {
7:     // Comment: Declares class Empty, part of the Empty HAL interface implementation.
8: public:
9:     // Comment: Sets the C++ member access level to public for following class members.
10: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
11:     bool give() override;
12:     // Comment: Declares an implementation of a virtual AP_HAL interface method.
13:     bool take(uint32_t timeout_ms) override;
14:     // Comment: Declares an implementation of a virtual AP_HAL interface method.
15:     bool take_nonblocking() override;
16:     // Comment: Declares an implementation of a virtual AP_HAL interface method.
17:     bool check_owner() { return true; }
18:     // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
19: private:
20:     // Comment: Sets the C++ member access level to private for following class members.
21:     bool _taken;
22:     // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++ interface.
23: };
24: // Comment: Closes a class declaration and terminates it with the required semicolon.

```

Storage.cpp

Path: ardupilot/libraries/AP_HAL_Empty/Storage.cpp

```

1: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include <string.h>
3: #include "Storage.h"
4: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: using namespace Empty;
6: // Comment: Brings the named namespace into local lookup so implementation code can use shorter type names.
7: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
8: Storage::Storage()

```

```

    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
8: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
9:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
10: void Storage::init()
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
11: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
12:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
13: void Storage::read_block(void* dst, uint16_t src, size_t n) {
    // Comment: Opens the body where this class, method, conditional, or loop is implemented.
14:     memset(dst, 0, n);
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL inte
    // rface.
15: }
    // Comment: Closes the current scope.
16:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
17: void Storage::write_block(uint16_t loc, const void* src, size_t n)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
18: {}
    // Comment: Provides an empty implementation body because the stub has no work to perform.
19:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.

```

Storage.h

Path: ardupilot/libraries/AP_HAL_Empty/Storage.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::Storage : public AP_HAL::Storage {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
    // Comment: Sets the C++ member access level to public for following class members.
7:     Storage();
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL inte
    // rface.
8:     void init() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
9:     void read_block(void *dst, uint16_t src, size_t n) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:    void write_block(uint16_t dst, const void* src, size_t n) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
11: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

UARTDriver.cpp

Path: ardupilot/libraries/AP_HAL_Empty/UARTDriver.cpp

```

1:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
2: #include "UARTDriver.h"
3: #include <AP_Common/ExpandingString.h>
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: Empty::UARTDriver::UARTDriver() {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    // rk here.
6:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
7: /* Empty implementations of virtual methods */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    // tes.
8: void Empty::UARTDriver::_begin(uint32_t b, uint16_t rxS, uint16_t txS) {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    // rk here.
9: void Empty::UARTDriver::_end() {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    // rk here.
10: void Empty::UARTDriver::_flush() {}
    // Comment: Defines an empty method body because the Empty HAL intentionally performs no platform wo
    // rk here.
11: bool Empty::UARTDriver::is_initialized() { return false; }

```

```

    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
12: bool Empty::UARTDriver::tx_pending() { return false; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
13:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
14: uint32_t Empty::UARTDriver::_available() { return 0; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
15: uint32_t Empty::UARTDriver::txspace() { return 1; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
16: bool Empty::UARTDriver::_discard_input() { return false; }
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
17: size_t Empty::UARTDriver::_write(const uint8_t *buffer, size_t size)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
18: {
    // Comment: Opens the body for the preceding declaration or control statement.
19:     return size;
    // Comment: Returns the computed or stored value required by the current AP_HAL method.
20: }
    // Comment: Closes the current scope.
21: ssize_t Empty::UARTDriver::_read(uint8_t *buffer, uint16_t size)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
22: {
    // Comment: Opens the body for the preceding declaration or control statement.
23:     return 0;
    // Comment: Returns a neutral zero value, typically meaning no data, no size, or no available hardware
    state.
24: }
    // Comment: Closes the current scope.
25:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
26: #if HAL_UART_STATS_ENABLED
    // Comment: Starts a preprocessor condition so this code is compiled only when the build configuration
    matches.
27: void Empty::UARTDriver::uart_info(ExpandingString &str, StatsTracker &stats, const uint32_t dt_ms)
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    interface.
28: {
    // Comment: Opens the body for the preceding declaration or control statement.
29:     str.printf("EMPTY\n");
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
30: }
    // Comment: Closes the current scope.
31: #endif
    // Comment: Ends the matching preprocessor conditional block.

```

UARTDriver.h

Path: ardupilot/libraries/AP_HAL_Empty/UARTDriver.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include "AP_HAL_Empty.h"
4:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
5: class Empty::UARTDriver : public AP_HAL::UARTDriver {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
6: public:
    // Comment: Sets the C++ member access level to public for following class members.
7:     UARTDriver();
    // Comment: Declares or calls a function; in this HAL it is usually part of the required AP_HAL interface.
8:     /* Empty implementations of UARTDriver virtual methods */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
9:     bool is_initialized() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
10:    bool tx_pending() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
11:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
12:    /* Empty implementations of Stream virtual methods */
    // Comment: Source comment from the original file; documents licensing, intent, or implementation notes.
13:    uint32_t txspace() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.

```

```

14: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
15: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
16: #if HAL_UART_STATS_ENABLED
    // Comment: Starts a preprocessor condition so this code is compiled only when the build configurati
    on matches.
17:     // request information on uart I/O for one uart
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
18:     void uart_info(ExpandingString &str, StatsTracker &stats, const uint32_t dt_ms) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
19: #endif
    // Comment: Ends the matching preprocessor conditional block.
20: // Comment: Blank separator line; keeps related declarations or statements visually grouped.
21: protected:
    // Comment: Sets the C++ member access level to protected for following class members.
22:     void _begin(uint32_t b, uint16_t rxS, uint16_t txS) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
23:     size_t _write(const uint8_t *buffer, size_t size) override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
24:     ssize_t _read(uint8_t *buffer, uint16_t size) override WARN_IF_UNUSED;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
25:     void _end() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
26:     void _flush() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
27:     uint32_t _available() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
28:     bool _discard_input() override;
    // Comment: Declares an implementation of a virtual AP_HAL interface method.
29: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

Util.h

Path: ardupilot/libraries/AP_HAL_Empty/Util.h

```

1: #pragma once
    // Comment: Prevents this header from being included more than once in a single translation unit.
2:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
3: #include <AP_HAL/AP_HAL.h>
4: #include "AP_HAL_Empty_Namespace.h"
5:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
6: class Empty::Util : public AP_HAL::Util {
    // Comment: Declares class Empty, part of the Empty HAL interface implementation.
7: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.

```

WSPIDevice.h

Path: ardupilot/libraries/AP_HAL_Empty/WSPIDevice.h

```

1: /*
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
2: * This file is free software: you can redistribute it and/or modify it
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
3: * under the terms of the GNU General Public License as published by the
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
4: * Free Software Foundation, either version 3 of the License, or
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
5: * (at your option) any later version.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
6: *
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
7: * This file is distributed in the hope that it will be useful, but
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
8: * WITHOUT ANY WARRANTY; without even the implied warranty of
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
9: * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no
    tes.
10: * See the GNU General Public License for more details.
    // Comment: Source comment from the original file; documents licensing, intent, or implementation no

```



```

tes.
11: *
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
12: * You should have received a copy of the GNU General Public License along
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
13: * with this program. If not, see <http://www.gnu.org/licenses/>.
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
14: *
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
15: * Code by Andy Piper and Siddharth Bharat Purohit
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
16: */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
17: #pragma once
// Comment: Prevents this header from being included more than once in a single translation unit.
18:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
19: #include <AP_HAL/HAL.h>
20: #include <AP_HAL/WSPIDevice.h>
21: #include <AP_HAL/utility/OwnPtr.h>
22:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
23: #ifndef HAL_USE_WSPI_DEFAULT_CFG
// Comment: Starts a preprocessor condition so this code is compiled only when the build configurati
on matches.
24: #define HAL_USE_WSPI_DEFAULT_CFG 1
// Comment: Defines a preprocessor symbol used to configure or guard later code.
25: #endif
// Comment: Ends the matching preprocessor conditional block.
26:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
27: namespace Empty
// Comment: Opens a namespace scope so these declarations are grouped under the HAL-specific namespa
ce.
28: {
// Comment: Opens the body for the preceding declaration or control statement.
29:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
30: class WSPIDevice : public AP_HAL::WSPIDevice
// Comment: Declares class WSPIDevice, part of the Empty HAL interface implementation.
31: {
// Comment: Opens the body for the preceding declaration or control statement.
32: public:
// Comment: Sets the C++ member access level to public for following class members.
33:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
34:     WSPIDevice() { }
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
interface.
35:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
36:     /* See AP_HAL::Device::transfer() */
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
37:     bool transfer(const uint8_t *send, uint32_t send_len,
// Comment: Supplies one argument in a multi-line constructor or function call.
38:                  uint8_t *recv, uint32_t recv_len) override
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
interface.
39:     {
// Comment: Opens the body for the preceding declaration or control statement.
40:         return false;
// Comment: Reports failure or absence of data for this stubbed hardware operation.
41:     }
// Comment: Closes the current scope.
42:
// Comment: Blank separator line; keeps related declarations or statements visually grouped.
43:     // Set command header for upcoming transfer call(s)
// Comment: Source comment from the original file; documents licensing, intent, or implementation no
tes.
44:     void set_cmd_header(const CommandHeader& cmd_hdr) override
// Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
interface.
45:     {
// Comment: Opens the body for the preceding declaration or control statement.
46:         return;
// Comment: Returns the computed or stored value required by the current AP_HAL method.
47:     }

```

```

    // Comment: Closes the current scope.
48:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
49:     AP_HAL::Semaphore* get_semaphore() override
    // Comment: Keeps the AP_HAL Empty implementation structurally compatible with the corresponding C++
    // interface.
50:     {
    // Comment: Opens the body for the preceding declaration or control statement.
51:         return nullptr;
    // Comment: Returns no object or no callback handle because the Empty HAL does not provide this hard
    // ware service.
52:     }
    // Comment: Closes the current scope.
53: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.
54:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
55: class WSPIDeviceManager : public AP_HAL::WSPIDeviceManager
    // Comment: Declares class WSPIDeviceManager, part of the Empty HAL interface implementation.
56: {
    // Comment: Opens the body for the preceding declaration or control statement.
57: };
    // Comment: Closes a class declaration and terminates it with the required semicolon.
58:
    // Comment: Blank separator line; keeps related declarations or statements visually grouped.
59: }
    // Comment: Closes the current scope.

```